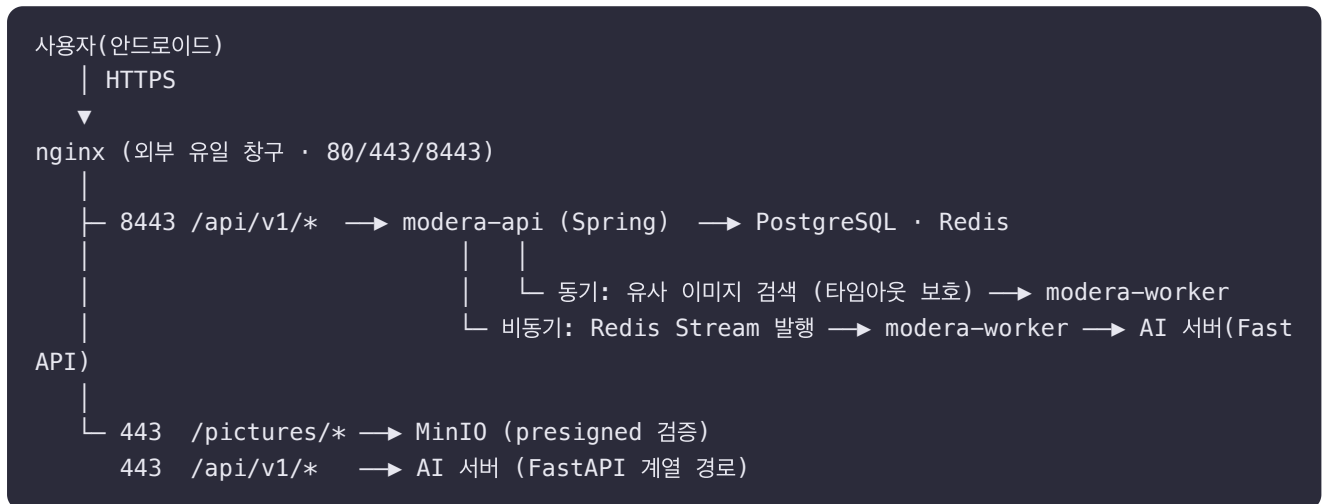


MODERA 빌드·배포 매뉴얼

기준일: 2026-08-10. 본 문서의 버전·포트·구성은 전부 운영 서버 실측값이다.

1. 시스템 개요

MODERA 백엔드는 AWS Lightsail 단일 인스턴스(4 vCPU / 15GB RAM, Ubuntu 24.04) 위에서 Docker Compose로 구동된다. 역할별로 3개 애플리케이션 서버로 분리되어 있다.



api와 worker 간 결합은 동기 1개(타임아웃 보호)와 비동기(Redis Stream)로 제한되며, AI 서버는 큐 뒤에 있어 사용자 요청 경로에서 격리된다. 컨테이너 간 통신은 Docker 네트워크 내부(컨테이너명 기반)로 이뤄지고, api-worker는 호스트에 포트를 열지 않는다.

2. 기술 스택 및 버전 (실측)

구성	버전	비고
Java (JVM)	21.0.11 (Eclipse Temurin LTS)	가상 스레드 사용
Spring Boot	4.0.2	backend/gradle.properties 의 SPRING_BOOT_VERSION. 내장 Tomcat
Gradle	9.3.0	Kotlin 2.2.21 · Groovy 4.0.29 포함
PostgreSQL	16.14	커스텀 이미지(2.1절)
└─ pgvector	0.8.5	유사 이미지 벡터 검색
└─ pg_bigm	1.2	텍스트 부분일치 인덱스(소스 빌드)
Redis	7.4.9 (7-alpine)	Streams 사용
nginx	1.27.5 (alpine)	변수 기반 업스트림
MinIO	latest	S3 호환 스토리지
Prometheus / Grafana / Loki / Promtail	latest	관측 스택
OpenSearch	2.17.0 (+nori 커스텀 이미지)	AI 파트 검색엔진
Docker / Compose	Docker 29.6.2 / Compose v5.3.1	
IDE	IntelliJ IDEA 2023.3.8 (Ultimate Edition)	SSAFY 지급 환경 기준

2.1 커스텀 이미지

이미지	Dockerfile 위치	내용
infra-postgres	(레포) infra/postgres/Dockerfile	pgvector/pgvector:pg16 베이스에 pg_bigm 1.2를 소스 빌드로 추가. 두 확장을 모두 포함한 공식 이미지가 없어 커스텀 구성
ai-opensearch-nori	(레포) ai/ai_main/opensearch.Dockerfile	opensearchproject/opensearch:2.17.0 베이스 + analysis-nori 플러그인 설치

레지스트리에 푸시하지 않는 로컬 빌드 이미지이므로, 신규 환경에서는 위 Dockerfile로 먼저 빌드해야 한다.

3. 저장소·서버 디렉터리 구조

레포의 infra/ 폴더가 인프라 구성의 단일 진실원이다. 서버 실물과 레포가 동기화되어 있다(2026-08-09 완료).

위치	내용
(레포) backend/	Spring 소스, api-server · analysis-worker · event-contract 모듈
(레포) ai/	AI 서버(FastAPI) 소스와 compose
(레포) infra/	인프라 compose(docker-compose.yml), nginx/conf.d/(리버스 프록시 설정), postgres/(Dockerfile·init), prometheus/ · loki/ · promtail/(관측 설정), app-spring/(앱 compose), deploy-api.sh, docker-compose.green.yml
(레포) infra/jenkins/	CI 파이프라인 정의, backend.Jenkinsfile(modera-backend 잡), ai.Jenkinsfile(modera-ai 잡)
(서버) /home/ubuntu/app/spring/	앱 compose 실행 위치. .env 는 서버에만 존재
(서버) ~/infra/	인프라 compose 실행 위치. .env 는 서버에만 존재
(서버) /home/ubuntu/app/AI/	AI compose 실행 위치. .env 는 Jenkins credential에서 배포 시 동기화

4. 환경 변수

전체 키 목록·용도·관리 방법은 05_환경변수_전체목록.md 참조. 요약:

파일	키 수	소비 주체
/home/ubuntu/app/spring/.env	14	modera-api, modera-worker
/home/ubuntu/infra/.env	10	PostgreSQL, MinIO 등 인프라
/home/ubuntu/app/AI/.env	약 50	ai-service (원본은 Jenkins credential ai-server-dotenv)
/opt/modera/secrets/firebase-service-account.json	파일	FCM 발송용 Google 서비스 계정

값은 서버의 실제 파일에만 존재하며 레포에는 커밋하지 않는다. 레포의 infra/.env.example, infra/app-spring/.env.example 이 키 목록 템플릿이다.

5. 데이터베이스 구성

단일 PostgreSQL 인스턴스(infra-postgres-1) 안에 서비스별 DB·계정 분리:

DB	소유 계정	사용 서버	확장
modera_api	api_admin	modera-api	vector 0.8.5, pg_bigm 1.2
modera_analysis	analysis_admin	modera-worker	vector 0.8.5, pg_bigm 1.2
modera	admin	(공용/초기)	plpgsql만

- 초기화: infra/postgres/init/ 스크립트가 최초 기동 시 실행됨 (docker-entrpoint-initdb.d)

- 데이터 볼륨: pgdata
- 접근: 호스트 127.0.0.1:5432 (SSH 터널 전용, 외부 비노출)
- 스키마 마이그레이션: 각 서버가 자기 DB의 Liquibase를 기동 시 실행

6. 포트 맵 (실측)

구분	포트	서비스
외부 노출	22	SSH
	80 / 443 / 8443	nginx (80은 리다이렉트, 443은 AI·MinIO 경로, 8443은 백엔드 API·Grafana)
호스트 로컬(127.0.0.1)	5432	PostgreSQL
	6379	Redis
	8000	AI 서버 (2026-08-09 외부 노출 차단, SSH 터널 전용)
	9000 / 9001	MinIO (API / 콘솔)
	9090	Prometheus
	9200 / 5601	OpenSearch / Dashboards (SSH 터널 전용)
내부망 전용(포트 비공개)	-	modera-api · modera-worker (infra_default 네트워크)

Docker 네트워크: infra_default(백엔드·인프라), ai_default(AI 파트), jenkins_default, shared-web(nginx·AI·api 간 브리지).

7. 로컬 개발 환경

- 로컬 인프라(PostgreSQL 2개·Redis·MinIO)는 레포의 backend/local-infra/ compose로 기동한다. 운영 인프라 구성은 레포 infra/ 폴더를 참조한다.
- DB는 커스텀 이미지 필요(2.1절). pgvector·pg_bigm 없이는 애플리케이션 기동이 실패한다.
- 기동 후 확인: `curl -s http://localhost:8080/actuator/health` 응답이 `{"status":"UP"}` (포트는 로컬 설정 기준)

8. CI/CD

8.1 Jenkins

- 잡 2개(단일 브랜치 Pipeline): modera-backend 는 develop/backend 브랜치의 infra/jenkins/backend.Jenkinsfile 을, modera-ai 는 develop/ai 브랜치의 infra/jenkins/ai.Jenkinsfile 을 읽는다.
- 트리거: GitLab 웹훅(push 즉시) + Poll SCM 3분 주기(웹훅 유실 대비 보조).
- 두 파이프라인 모두 changeset 조건(backend/, ai/)을 사용해 해당 파트 변경 시에만 빌드·배포 단계가 실행된다.

8.2 무중단 배포 (api 한정 blue-green 바운스)

api 컨테이너만 무중단으로 교체하며 worker-AI는 대상이 아니다. 절차(infra/deploy-api.sh):

1. green(후보) 컨테이너 기동: docker-compose.green.yml (본체 서비스를 extends 상속)
2. 헬스 게이트: 관리 포트 9080 /actuator/health UP 확인까지 대기
3. nginx 업스트림 스왑: conf의 변수 \$api_upstream 을 green으로 변경 후 nginx -t && reload
4. 드레인 130초: 기존 요청 소진 대기, Prometheus 5xx 지표로 이상 판정(임계 초과 시 자동 원복)
5. 본체(modera-api)를 신규 이미지로 재생성, 트래픽 복귀, green 정리

수동 롤백: 3단계의 변수를 원래 값으로 되돌리고 nginx -t && reload.

9. 배포·운영 특이사항 (트러블슈팅)

1. nginx conf 수정 규칙: 반드시 `nginx -t` 와 `docker exec infra-nginx-1 nginx -s reload` 를 한 묶음으로 실행한다. 변수+resolver 패턴은 -t 가 통과해도 요청 시점에 실패할 수 있으므로(업스트림 컨테이너 부재 시 502) conf 수정 후 대표 경로 curl 검증이 필수다: `curl -sk https://localhost:8443/actuator/health` , AI 경로 와 /pictures/ 경로 각 1회.
2. 커스텀 PostgreSQL 이미지: 신규 환경에서는 infra/postgres/Dockerfile 빌드가 선행되어야 한다. pgvector·pg_bigm 확장은 DB별로 CREATE EXTENSION 필요(init 스크립트가 수행).
3. latest 태그: MinIO와 관측 스택이 latest 태그 사용 중이다. 재현성을 위해 이관 시 태그 고정을 권장한다.
4. AI에서 백엔드 별칭 의존: AI 서버는 SPRING_BASE_URL=http://spring-api:8080 으로 백엔드를 호출하며 spring-api 는 shared-web 네트워크의 별칭이다. 이 별칭은 앱 compose의 modera-api 서비스에 선언되어 있어(aliases: [spring-api]) 배포마다 자동 유지되고, green 도 extends 상속으로 같은 별칭을 받아 배포 중에도 AI 호출이 끊기지 않는다.
5. AI .env 관리: 서버의 /home/ubuntu/app/AI/.env 는 매 AI 배포 때 Jenkins credential(ai-server-dotenv)로 덮어써 진다. 값 수정은 반드시 Jenkins credential에서 할 것.

10. 기동 순서 (신규 환경 구축 시)

1. 레포 클론 후 infra/postgres/Dockerfile 빌드, 인프라 compose 기동 (infra/ 위치에서 `docker compose up -d`)
2. PostgreSQL init 완료 확인, 앱 .env 작성(05 문서 참조), firebase-service-account.json 배치
3. 앱 compose 기동 (infra/app-spring/docker-compose.yml 기준, `docker compose up -d`)
4. AI compose 기동 (ai/ai_main/docker-compose.yml, AI .env 작성 후)
5. 검증: `curl -sk https://localhost:8443/actuator/health` 응답 UP, 대표 API 1회 호출, Grafana 대시보드 수집 확인